Look-See-Do Fighter

AI 협업을 통한 격투 게임 리팩터링 및 개발 자동화

개발을 중단했던 2D 온라인 격투 게임을 다시 분석하고, 기존 전투 콘텐츠를 보존하면서 결정론적 아키텍처와 게임플레이를 개선했습니다.

Codex와 Unity MCP를 활용해 코드 조사부터 기능 구현, 테스트 및 검증 결과 관리까지 이어지는 AI Native 개발 워크플로를 구축했습니다.

결정론적 네트워크 · 기존 시스템 리팩터링 · AI 개발 워크플로 · 자동 검증

TECH STACK

Game Engine	Unity 6 / C#
Networking	Photon Quantum 3
Architecture	Deterministic ECS / Predict & Rollback
AI Development	Codex CLI / Astra / Unity MCP
Automation	AGENTS.md · Skills · Hooks · Git/GitHub
Testing	Unity Test Framework / NUnit / Python

프로젝트 재개와 개발 목표

기존 게임의 완성과 함께, 반복 가능한 AI Native 개발 방식을 구축했습니다.

재개 시점의 과제

전투 콘텐츠는 유지하고
설계 경계는 다시 정리

기본 전투·공격 기술·애니메이션·네트워크 구조가 구현된 프로젝트였습니다.

공유 에셋과 로컬 데이터에 의존한 실행 상태, 공격별 중복 코드, 복잡한 전이와 반복적인 수동 검증이 유지보수 부담으로 이어졌습니다.

01 구조적 안정성

Quantum Simulation과 Unity View 책임 분리

02 기존 콘텐츠 보존

37종 공격 타격 시점·애니메이션 참조 유지

03 게임플레이 확장

선입력·3선승·공중 콤보·전투 연출

04 AI 개발 체계

Skills·Hooks·MCP·테스트·버전 관리 표준화

개발 접근 방식

분석 → 결정론 구조 → 콘텐츠 리팩터링 → 기능 확장 → 플레이 오류 해결 → 검증·기록

기술 구성과 프레임 단위 심리전

사용할 기술을 직접 구성하고, 발동·지속·회복과 거리·타이밍의 상호작용을 읽는 게임입니다.

28 방향 7 × 버튼 4 슬롯 **3** 슬롯별 기술 선택지 **140** 기본 로드아웃 예산

기술 커스터마이징

기술별 비용과 예산 제한 안에서 28슬롯의 로드아웃을 구성합니다.

프레임 기반 전투

발동·지속·회복, 상·중·하단 판정과 가드·패링·카운터·경직을 이용해 거리와 공격 타이밍을 선택합니다.

온라인 매치

Photon 입력 동기화 · 라운드 HP/전투 상태 초기화
3승 선취 · KO/동시 KO · 최종 결과

GAMEPLAY CAPTURE

기술 선택 화면 또는 실제 대전 화면을 삽입해 주세요.

제한된 기술 구성과 프레임 단위 선택이
전략적 깊이를 만드는 게임을 지향합니다.

Simulation과 Unity View의 책임 분리

동일한 입력과 상태로 같은 전투 결과를 계산하도록 ECS·Frame 경계를 정리했습니다.



상태와 데이터

- 전투 결과에 영향을 주는 지속 상태는 Frame
- 공격 설정은 불변 데이터로 취급
- 판정은 FP·정수 tick 기반 계산

판정과 표현

- 시각적 상태가 전투 결과를 결정하지 않음
- Quantum Animator·시스템 실행 순서 보존
- 롤백 복원 상태와 화면 표현 상태를 구분

복원해야 하는 상태의 소유권을 명확히 하여 네트워크 동기화와 기능 확장에 대응했습니다.

로컬 선택·공유 설정·Frame 실행 상태

플레이어별 상태가 입력 계약과 Frame 복원 경계를 우회하던 구조를 바꿨습니다.

PROBLEM

스폰이 각 클라이언트의 PlayerPrefs에서 기술을 직접 읽었습니다.

잽 연계 예약 BufferedNextAttack은 공유 Window Event 에셋에 저장돼 있었습니다.

실행 상태의 소유권

공유 설정과 플레이어 상태,
호출별 임시 계산을 분리했습니다.

로드아웃 전달 구조

PlayerPrefs → 검증 → RuntimePlayer → AddPlayer → Frame

- 기술 ID·슬롯 수·비용을 검증한 복사본 전달
- Simulation은 f.GetPlayerData()로 접근
- 반복 AddPlayer의 중복 생성·상태 초기화 방지

- BufferedNextAttack → 플레이어 Frame 컴포넌트
- 호출별 임시 상태 → 지역변수
- 공격 수치·설정 → 불변 AssetObject
- 애니메이션 프레임 환산 → FP 계산

후속 검사: 서로 다른 로드아웃·반복 AddPlayer·입력 재생·Frame 복원 시나리오 확인

37종 공격의 판정 계약 보존

타격 데이터와 생성 절차를 분리하고, 공격별 고유 동작은 기존 이벤트에 유지했습니다.

PROBLEM → SOLUTION

피해·경직·피격 높이·생성 코드가 중복됐지만, 단순 공통화는 타이밍·특수 동작·Animator·직렬화 참조를 바꿀 위험이 있었습니다.

대표 유형부터 점진적으로 이관

1 윈어퍼 → **4** 잽 → **2** 다단히트 → **30** 나머지

중복 생성 절차를 공통화하면서 기존 게임플레이 계약을 유지했습니다.

구성 요소	담당 역할
LSDFSingleHitDefinition	단타 피해·경직·판정 데이터
LSDFTimedHitDefinition	다단 생성 프레임·타격 데이터
LSDFHitboxFactory	Collider·Hitbox·파괴 tick
기존 Window Event	이동·연계·특수 동작·종료

보존한 계약

생성 프레임·피해·경직 / 타입명·GUID·Asset ID·Animator
다단 타이밍·특수 처리 / 좌우 반전·생성/파괴 tick 경계

프레임 단위 선입력 시스템

여러 tick을 거치는 Animator 전이에서 입력이 사라지지 않도록 대기와 전달 상태를 나눴습니다.

입력 캡처 새 눌림의 방향·버튼·tick → **대기** 행동 잠금 중 최신 명령 1개 → **전달 중** Animator 소비까지 유지 → **정리** 만료·KO·라운드 전환

동작과 경계

- 잠금 해제 후 첫 명령 처리 tick에 전달
- Animator가 처리할 때까지 승인 명령 유지
- 홀드로 유효시간이 계속 갱신되지 않음
- 방향 변경·키 해제 후에도 저장 명령 유지
- 유효시간 초과·KO·라운드 전환 시 정리

설정과 실제 관측

20 tick
현재 게임 설정

10 tick
새 객체 초기값

입력 384 → 전달 392 tick
Photon 양쪽 세션에서 정상 전달 관측

후속 전체 검사 157 PASS · 8tick 차이는 해당 잠금 사례의 관측값이며 평균 입력 지연이 아닙니다.

KO 이후 3선승 매치 흐름 연결

라운드 종료·초기화·다음 시작·최종 종료의 기준 상태를 Quantum Frame으로 관리합니다.

WaitingForPlayers → **Intro** → **Fighting** → **RoundEnd** → **Intro / MatchEnd**

주요 구현

- KO 결과를 한 번만 확정해 중복 처리 방지
- 동시 KO는 무득점, 같은 라운드 재시작
- HP·위치·공격·버퍼·콤보 상태 초기화
- 승수·기술 로드아웃은 라운드 사이 유지
- 결과 120tick · 인트로 150tick · 3승 선취
- verified 매치 종료 → 연결 종료·메인 복귀

MATCH RESULT CAPTURE

최종 결과 또는 Round/Fight 전환 화면을 넣어 주세요.

Photon: KO 5회·초기화/인트로 4회·최종 3:2·verified Frame 진행 확인. 메인 복귀는 후속 구현·사용자 확인.

애니메이션 진입 수와 실제 타격 수

다른 의미의 카운터를 피해 공식에 사용하던 문제를 실제 콤보 접촉 횟수 기준으로 수정했습니다.

ROOT CAUSE

LSDF_Player.hitCount

실제 적중 수가 아닌 Air 애니메이션 진입 횟수였습니다. 피격 전이가 바뀌면서 두 값이 달라졌고, 특정 카운트에서 피해 0 분기가 반복됐습니다.

↓

LSDF_AirCombo.HitCount

Juggle.OnHit → 실제 HitCount 갱신 → ApplyHit 계산. 정수 비율과 곱셈 중간 오버플로를 고려했습니다.

타격	배율	기본 피해 20
지상 시동기	100%	20
공중 후속 1타	70%	14
공중 후속 2타	50%	10
공중 후속 3~5타	30%	6

109 PASS / 1 FAIL

110개 중 체력 단위 32개 PASS. 통합 예상값 보정 후 재실행하지 않은 범위는 미검증으로 기록했습니다.

데이터가 실제로 무엇을 의미하는지 확인하는 것이 원인 분석의 핵심이었습니다.

공중 상태와 자연스러운 타격 반응

고정 높이의 부자연스러운 움직임과, 화면상 재상승 현상을 서로 다른 원인으로 분리했습니다.

RehitRising → Returning → **SlowFalling** → Natural Fall

Frame 기반 궤적

- 단계·기준 높이·시작/종료 tick 저장
- 재타격 작은 상승·기준 높이 복귀
- Hermite 보간·tick 기반 자연 낙하 가속
- 5후속타·지정 마무리의 별도 날리기
- 공격자 반대 방향 밀림, 거리·시간 Inspector

Sprite pivot 원인 분리

물리 높이와 표시 기준점을 비교해 원인을 확인했습니다. 양쪽 공중 애니메이션 후반부의 메타데이터 60개를 통일했습니다.



원본 동작 에셋 · 실행 화면/pivot 전후 비교 아님

106 PASS
자연 낙하

99 PASS
공중 밀림

66 PASS
Sprite pivot

제어된 입력 재생·Frame 복원
체크섬 비교 수행

판정과 분리된 전투 피드백

게임 결과를 바꾸지 않으면서 예측 표현의 생성·확인·취소 수명을 관리했습니다.

이펙트와 예측 이벤트

- 경기 updater 소유 ViewComponent로 연결
- 타격 유형별 프리팹·Animator 주기 기반 수명
- 예측 이벤트 취소 시 VFX·오디오 정리
- MaterialPropertyBlock 기반 색상 표현

체력과 카메라 연출

- HP 변화에 따른 체력바 흔들림
- 기술별 카운터 줌 설정
- Follow → Zoom → Shake 변환 합성
- 중복 이벤트·잔여 오프셋 방지

COMBAT FEEDBACK CAPTURE

피격 순간의 이펙트와 HUD·카메라 구도가 보이는 화면

화면 상태의 기준

HP: predicted / 승패·라운드: verified
예측 시각효과와 확정 결과를 구분

아트를 조정할 수 있는 Unity View 도구

카메라 투영·배경 재사용·HUD 원본 비율을 고려하고 Inspector에서 조절할 수 있게 했습니다.

무한 스테이지 배경

배경 깊이를 보정해 캐릭터와 줌 배율을 맞췄습니다.
카메라별 SpriteRenderer 3개를 고정 재사용하고,
화면 밖 타일을 반대편으로 이동합니다.

큰 줌아웃은 Tiled 범위 확장, 무제한 객체 생성 방지.
시각 배경과 Quantum 충돌 지형의 책임을 분리했습니다.

이미지 기반 HUD

체력 배경·채움, Round1~5·Fight·Win/Lose와 승리 표시.
Inspector에서 Sprite·크기·미리보기·Undo를 지원하고,
원본 종횡비와 이미지의 상대 크기를 유지했습니다.

체력바 Sprite 반전

감소 방향·Scale·fillOrigin은 유지했습니다.
전체 Sprite atlas UV의 X만 반사했습니다.

작업별 실행 결과 · 합산하지 않음

배경·Photon 30 PASS / HUD 39 PASS
이미지 비율 22 PASS / UV 28 PASS

HUD / INSPECTOR CAPTURE

실제 HUD 또는 Sprite·크기 설정 Inspector를 넣어 주세요.

제약과 근거를 공유하는 AI 개발 환경

AI가 프로젝트 제약을 이해하고 검증 가능한 결과를 만들도록 작업 지식을 파일로 유지했습니다.

구성 요소	프로젝트에서의 활용
Codex CLI	코드 분석·구현·리팩터링 협업
Unity MCP	Editor·Console 조사·테스트 실행
AGENTS.md	프로젝트 규칙·기술 제약 관리
Skills	작업별 절차·검증 기준 표준화
SessionStart Hook	시작·재개 시 규칙·도구 진입점 안내
Python Guard	Simulation 위험 패턴 정적 검사
Git / GitHub	변경 관리·검토·게시 절차

프로젝트 지식의 외부화

AGENTS.md

개발 제약·공통 규칙

docs/ai/current-state.md

현재 상태·미완료 작업

docs/ai/development-workflow.md

구현·검증 절차

docs/ai/evidence/

실행 결과·검증 근거

새 세션에서도 최신 문서를 실제 Git·소스와 대조해 작업을 이어갑니다. 같은 설명을 반복하는 부담을 줄였습니다.

프로젝트 전용 절차와 자동 진입점

Skill은 작업 절차, Hook은 이벤트 진입점, 테스트는 실제 동작 검증을 담당합니다.

Skill	주요 책임
lsdf-game-development	기능·버그·UI·밸런스
lsdf-quantum-refactor	상태 소유권·구조 리팩터링
lsdf-quantum-verify	CodeGen·컴파일·재생 검증
lsdf-portfolio-evidence	기술 성과·검증 근거 정리
publish-unity-pr	Git 변경 검토·게시 준비

현재 등록: SessionStart

시작·재개·압축 시 프로젝트 루트를 찾고
공통 지침·최신 상태·검사기 진입점을 안내합니다.
실제 코드와 최신 문서를 대조해 재개합니다.

quantum_guard.py

- PlayerPrefs 등 로컬 데이터 직접 접근
- FP → float·비결정론 연산 패턴
- 판정 영역의 부적절한 Unity API
- 시간·난수·스레딩·공유 가변 상태

5개 Skill로 개발 작업과 검증 기준을 표준화했습니다. 정적 패턴 검사 통과는 의미적 결정론 증명이 아닙니다.

실행 결과와 후보 변경을 연결하는 절차

요구사항 → 조사 → 구현 → CodeGen/컴파일 → 정적 검사 → Unity 테스트 → 기록 → 승인 범위의 Git 게시

대표 검증 범위	실행 결과
선입력·매치	157 PASS
로드아웃·체력 연출	235 PASS
자연 재타격 낙하	106 PASS
공중 피격 밀림	99 PASS
Sprite pivot	66 PASS
배경·Photon 후속	30 PASS

서로 다른 시점의 실행 결과입니다. 중복 시나리오가 있으므로 합산하지 않습니다.

Unity MCP

refresh·컴파일·Console 오류/경고 확인, Scene·Asset·Animator 참조 분석, CodeGen·Unity 테스트 결과 수집, Photon 연결·Frame 복원을 수행했습니다.

GitHub 게시 후보 검토

- 기존 저장소의 브랜치·변경 확인
- 관련 파일만 staging, 비밀·대형 파일·이력 검사
- 테스트 후보와 커밋 대상 tree 일치 확인
- publish-unity-pr · 승인 범위의 Commit/Push

최근 게시에서 후보·커밋 tree의 일치와 원격 SHA 일치를 확인했습니다.

실패 후 수정해도 재실행하지 않았다면 미검증으로 기록합니다. 미실행을 PASS로 바꾸지 않습니다.

게임 개발 지식과 AI 운영을 함께 확장

문제를 정의하고 기술적 제약을 전달하며, AI의 결과를 검증할 수 있는 판단력을 쌓았습니다.

01 결정론적 시스템 설계

로컬·공유 설정·Frame 분리, 기존 공격 보존

02 게임플레이 문제 해결

입력·매치·공중·전이·표현의 원인 분석

03 AI 개발 환경 구축

AGENTS·Skills·Hook·MCP·Guard와 인계

04 검증·변경 이력 관리

실행 근거·한계 기록, Git 후보·상태 추적

남은 개선 과제

- 일부 공격 이벤트·미검증 테스트 정리
- 강제 지연·지터 환경의 rollback 검사
- 독립 빌드·다중 기기·배포 검증
- GitHub Actions CI·자동 PR 확장
- 동일 조건 성능 측정·효과 정량화

AI가 안정적으로 개발할 수 있는 환경을 설계하는 게임 개발자